B.M.S. COLLEGE OF ENGINEERING
(Autonomous College under VTU)
Bull Temple Road, Basavangudi, Bangalore – 560019

DEVOPS LAB REPORT

BACHELOR OF ENGINEERING

in

Computer Science & Engineering – Data Science

Submitted by

Ullas N	**1BM23CD067**
H Sampreeth Bhat	**1BM23CD020**
R V Abhishek	**1BM23CD047**

Under the Guidance of

Sandhya G

Department of CSE – Data Science
B.M.S. College of Engineering

AY 2025–2026

CERTIFICATE

This is to certify that the DevOps Lab report titled *"Real-Time Network Intrusion Detection System using NoSQL Databases and CI/CD Pipeline"* submitted by **Ullas N (1BM23CD067)**, **H Sampreeth Bhat (1BM23CD020)**, and **R V Abhishek (1BM23CD047)** is a bonafide record of work carried out under my supervision during the academic year 2025–2026, in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science & Engineering (Data Science) at B.M.S. College of Engineering, Bangalore.

Sl. No.	Name	USN
1	Ullas N	1BM23CD067
2	H Sampreeth Bhat	1BM23CD020
3	R V Abhishek	1BM23CD047

Guide: Sandhya G **HOD**

Date: April 2026

DECLARATION

We, **Ullas N (1BM23CD067)**, **H Sampreeth Bhat (1BM23CD020)**, and **R V Abhishek (1BM23CD047)**, hereby declare that this DevOps Lab report titled *"Real-Time Network Intrusion Detection System using NoSQL Databases and CI/CD Pipeline"* is a bonafide record of work carried out by us under the guidance of **Sandhya G**. This work has not been submitted elsewhere for any degree or examination.

Sl. No.	Name	USN
1	Ullas N	1BM23CD067
2	H Sampreeth Bhat	1BM23CD020
3	R V Abhishek	1BM23CD047

Signature of Students **Signature of Guide**

Date: April 2026

Contents

A Next Gen Databases (NGD) **1**

1 Introduction **3**
1.1 Project Title . . . 3
1.2 Objective . . . 3
1.3 Problem Statement / Use Case . . . 3
1.3.1 Problem Statement . . . 3
1.3.2 Use Case . . . 4

2 Selection and Justification of NoSQL Database **6**
2.1 Types of NoSQL Databases Used . . . 6
2.2 Reason for Selection . . . 6
2.2.1 Apache Cassandra . . . 6
2.2.2 Redis . . . 6
2.3 Why It Suits the Use Case . . . 7
2.4 Brief Comparison with Relational Database . . . 7

3 Data Model and Schema Design **8**
3.1 Data Model Description . . . 8
3.1.1 Apache Cassandra – Wide-Column Store . . . 8
3.1.2 Redis – Key-Value Store . . . 8
3.2 Structure / Attributes . . . 8
3.2.1 Cassandra `nids.alerts` Table . . . 8
3.2.2 Cassandra `nids.sessions` Table . . . 9
3.2.3 Redis Key Patterns . . . 10
3.3 Relationships / Organisation . . . 10
3.4 Schema Diagram . . . 11

4 Implementation and Query Processing **12**
4.1 Structure Creation . . . 12
4.1.1 Creating the Keyspace . . . 12
4.1.2 Creating the Alerts Table . . . 12
4.1.3 Creating the Sessions Table . . . 13
4.2 Data Insertion and Update . . . 13

4.2.1 Inserting Alert Records into Cassandra . . . 13
4.2.2 Registering a New Session . . . 14
4.2.3 Caching Alerts in Redis . . . 14
4.2.4 Data Flow from Application to Database . . . 15
4.3 Data Retrieval and Query Execution . . . 15
4.3.1 Retrieve Latest Alerts by Session (Cassandra) . . . 15
4.3.2 Filter Alerts by Attack Type (Cassandra) . . . 16
4.3.3 List All Sessions (Cassandra) . . . 16
4.3.4 Delete a Session (Cassandra) . . . 16
4.3.5 Retrieve Recent Alerts from Redis (Live Feed) . . . 16
4.3.6 Retrieve Pre-Aggregated Statistics (Redis) . . . 17

5 Performance and Design Awareness **18**
5.1 Indexing . . . 18
5.1.1 Cassandra Primary Key as Built-In Index . . . 18
5.1.2 Redis List as Priority Queue . . . 18
5.2 Query Efficiency . . . 18
5.3 Scalability . . . 18
5.4 Optimisations Applied . . . 19

6 Results and Conclusion **20**
6.1 ML Model Performance . . . 20
6.2 Output Screenshots . . . 20
6.3 Key Observations . . . 21
6.4 Insights Gained . . . 21
6.5 Learning Outcomes . . . 22
6.6 Advantages of the Chosen NoSQL Databases . . . 22

B DevOps Pipeline **23**

7 Introduction **25**
7.1 Objective of the Pipeline . . . 25
7.2 Application Used . . . 25

8 Pipeline Design and Workflow **26**
8.1 Tools Used . . . 26
8.2 Pipeline Stages . . . 26
8.3 Jenkinsfile Structure . . . 27

9 Continuous Integration (CI) Process **30**
9.1 How CI is Implemented . . . 30
9.2 Trigger Mechanism . . . 30

9.3 Build and Test Automation . . . 30
9.3.1 Test Design Philosophy . . . 30
9.3.2 Verified Build Result: Build #7 . . . 31

10 Continuous Deployment (CD) Process **32**
10.1 Deployment Steps . . . 32
10.2 Deployment Environment . . . 32
10.3 Level of Automation . . . 32

11 Execution and Output Demonstration **34**
11.1 Build and Test Output . . . 34
11.2 Deployment Output . . . 34

12 Challenges and Solutions **35**
12.1 Challenge 1 – Deploy Stage Silently Skipped . . . 35
12.2 Challenge 2 – Windows Port-Binding Blocked by Firewall . . . 35
12.3 Challenge 3 – Tests Required Live Infrastructure . . . 36

13 Conclusion **37**
13.1 Learnings from the DevOps Pipeline . . . 37
13.2 Importance of CI/CD . . . 37

14 References **38**

List of Figures

1.1 High-level system architecture of the Real-Time NIDS 4

3.1 Cassandra and Redis data model for the NIDS project 11

4.1 End-to-end data flow from CSV source to Flask dashboard 15

6.1 NIDS Live Dashboard — real-time severity doughnut chart (138 alerts: 38 high, 89 medium, 11 low), session info card, and live alert feed 20

6.2 Alerts Page — filterable and sortable table showing attack type, severity badge, confidence score, source and destination 21

6.3 Analytics Page — bar chart of attack-type distribution (Generic → Fuzzers → Exploits leading) and alert timeline 21

6.4 Settings Page — session management with two sessions listed, service health indicator showing storage CONNECTED 21

8.1 Jenkins CI/CD pipeline flowchart for the NIDS project 27

8.2 Jenkins Blue Ocean pipeline view for Build #7 — all six stages green (Checkout → Install Dependencies → Lint → Test → Build Docker Image → Deploy), overall result: SUCCESS in 5 min 8 sec 29

9.1 Jenkins `nids-pipeline` status page — Build #7 successful (green badge), build artefacts (`integration-results.xml`, `test-results.xml`), and Latest Test Result showing **no failures**. Right panel shows Test Result Trend and Code Coverage Trend graphs across builds #1–#7. 31

9.2 Jenkins Build #7 detail page — started by SCM change, took 5 min 8 sec, Tests: no failures, Coverage: Line 57.50% (+3.09%), triggered by commit *refactor: simplify app initialisation in app.py*. 31

10.1 Jenkins console output for the Deploy stage — `docker compose ps` confirms all four services (`nids_app`, `nids_cassandra`, `nids_kafka`, `nids_redis`) are Up and healthy. Final lines show "Deployment successful! Dashboard: `http://localhost:5000`" and BUILD SUCCEEDED. 33

11.1 Jenkins dashboard showing the `nids-pipeline` job: Last Success Build #7 (9 min 30 sec ago), Last Failure Build #6 (18 min ago), last build duration 5 min 8 sec — demonstrating a series of iterative builds culminating in a stable green pipeline. 34
11.2 Jenkins Git Polling Log for the `nids-pipeline` job — Poll SCM is configured to run every minute (`* * * * *`), detecting commits to the GitHub repository and auto-triggering builds on new pushes. 34

List of Tables

2.1 NoSQL vs. RDBMS comparison for the NIDS use case 7

3.1 Column definitions for `nids.alerts` 9
3.2 Column definitions for `nids.sessions` 9
3.3 Redis key patterns and data types 10

5.1 Query efficiency analysis for core NIDS operations 18

6.1 Machine learning model performance on the UNSW-NB15 test set 20
6.2 Advantages demonstrated by Cassandra and Redis in the NIDS 22

8.1 DevOps tools used in the NIDS CI/CD pipeline 26

9.1 Jenkins Build #7 stage-by-stage results (fully green) 31

10.1 Services in the Docker Compose deployment stack 32

ABSTRACT

This report presents a real-time Network Intrusion Detection System (NIDS) that combines streaming analytics, machine learning inference, and NoSQL data management in a production-style DevOps workflow. The system ingests UNSW-NB15 traffic records through Apache Kafka, performs two-stage inference using Gradient Boosted Trees (binary attack detection) and Random Forest (attack-type classification), and stores outputs in a Cassandra + Redis dual database architecture for durable persistence and low-latency dashboard reads.

The implementation is integrated with a Jenkins-driven CI/CD pipeline that automates linting, testing, image builds, and branch-gated deployment using Docker Compose. The design demonstrates how model-serving, stream processing, NoSQL storage, and deployment automation can be combined into a reproducible and scalable cybersecurity analytics platform suitable for academic and industry-aligned DevOps practice.

Part A

Next Gen Databases (NGD)

This part explains the storage architecture of the NIDS platform, including the rationale for selecting Cassandra and Redis, schema design decisions, query patterns, and performance-oriented data operations.

Chapter 1
Introduction

1.1 Project Title

Real-Time Network Intrusion Detection System (NIDS) using NoSQL Databases and Machine Learning.

1.2 Objective

The objective of this project is to design and implement a real-time network intrusion detection system that:

- Ingests high-velocity network traffic records streamed through Apache Kafka.
- Performs **two-stage machine learning inference**:
  1. **Binary classification** (Gradient Boosted Trees, GBT) to determine whether each record is *Normal* or an *Attack*.
  2. **Multiclass classification** (Random Forest) to identify the specific attack category among nine types.
- Persists every intrusion alert in **Apache Cassandra** for durable, fault-tolerant long-term storage.
- Caches live alerts in **Redis** for sub-millisecond dashboard reads and real-time pub/sub notifications.
- Exposes a **Flask-based web dashboard** with live severity gauges, filterable alert tables, and analytics charts for security analysts.

1.3 Problem Statement / Use Case

1.3.1 Problem Statement

Modern enterprise and campus networks generate millions of traffic records per minute. Traditional signature-based Intrusion Detection Systems (IDS) maintain a fixed list of known attack signatures and fail against novel or zero-day exploits. Relational database management systems (RDBMS) are ill-suited for this streaming workload because:

- **Schema rigidity:** Adding new protocol fields requires `ALTER TABLE`, causing downtime.

- **Write bottlenecks:** SQL `INSERT` operations acquire row-level locks and incur B-tree index overhead at high insert rates.
- **No horizontal scaling:** RDBMS instances scale vertically (larger machines), which is expensive and finite.
- **JOIN overhead:** Analytical queries joining alerts with session metadata become prohibitively slow at millions of rows.

1.3.2 Use Case

A university or enterprise Network Operations Centre (NOC) uses this system to:

1. Continuously stream network traffic through the Kafka pipeline at up to 100 records per second (scalable to millions).
2. Automatically flag and classify suspicious traffic in real time via trained ML models with no human intervention.
3. Query historical alerts by session, time range, or attack category through the Flask REST API and dashboard.
4. Receive instant live notifications the moment a high-severity threat (DoS, Exploits, Shellcode, Backdoor) is detected via Redis pub/sub.

The system uses the **UNSW-NB15 dataset** from the University of New South Wales Canberra—a benchmark network security dataset with 2.5 million raw traffic records and a standard train/test split of 175,341 / 82,332 records used for model training and evaluation, with 42 feature columns and 9 distinct attack categories.

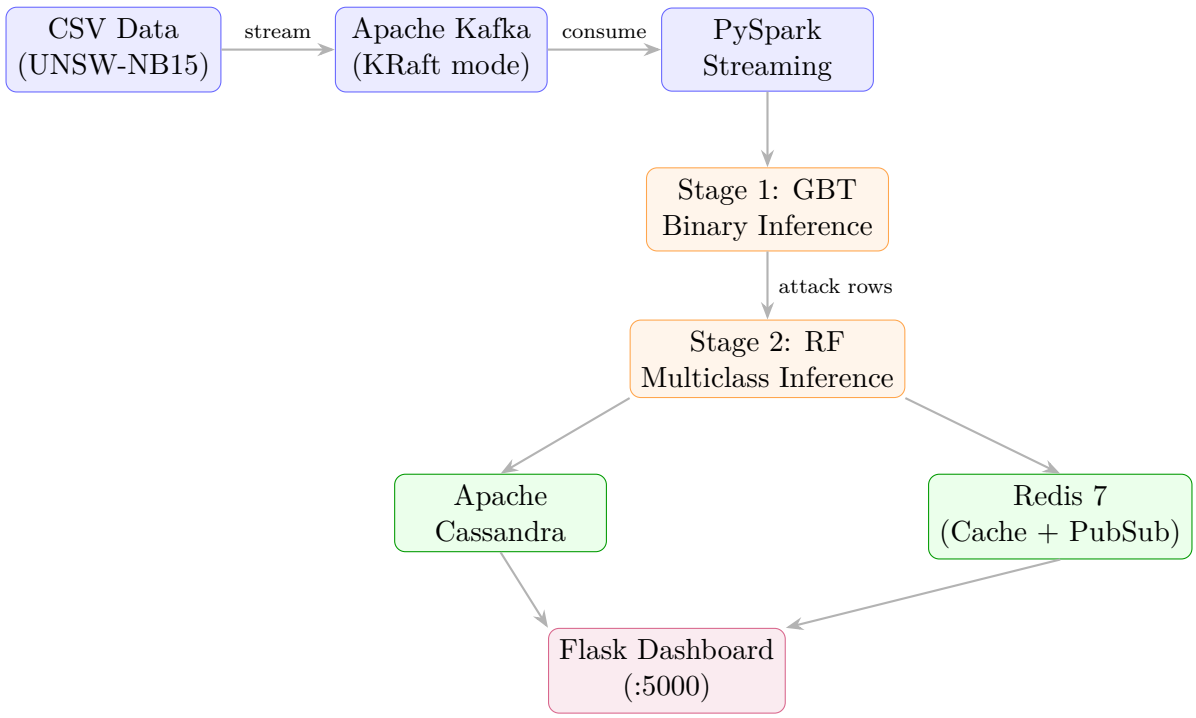
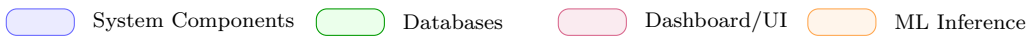


Figure 1.1: High-level system architecture of the Real-Time NIDS


System Components
Databases
Dashboard/UI
ML Inference

Chapter 2

Selection and Justification of NoSQL Database

2.1 Types of NoSQL Databases Used

This project employs **two complementary NoSQL databases**, each serving a distinct role:

1. **Apache Cassandra 4.1**—a **Wide-Column Store** used for durable, partitioned alert persistence (source of truth).
2. **Redis 7**—a **Key-Value Store** (with list, hash, and pub/sub support) used for in-memory caching and real-time event delivery (speed layer).

2.2 Reason for Selection

2.2.1 Apache Cassandra

- Designed for **high-write-throughput**: uses an LSM-tree (Log-Structured Merge-tree) storage engine with no row locks—every write is a sequential append.
- **Partition key model** distributes session-specific alert data across cluster nodes automatically via consistent hashing.
- **Clustering key** stores rows pre-sorted in reverse-chronological order on disk, making "latest N alerts" queries free (no sort step at query time).
- **Linear horizontal scalability**: adding nodes increases throughput proportionally without re-sharding.
- CQL (Cassandra Query Language) provides a familiar SQL-like syntax for DDL and DML operations.

2.2.2 Redis

- In-memory data structure store guarantees **sub-millisecond read latency**—essential for a dashboard that auto-refreshes every 5 seconds.
- The **LPUSH / LRANGE** list primitives maintain an ordered recent-alerts cache without application-level sorting.

- The built-in **pub/sub channel** (`alert_notifications`) allows the Spark pipeline to push live events to subscribed clients without polling.
- Acts as a **speed layer**: absorbs all dashboard read traffic, protecting Cassandra from repeated identical queries.
- Configurable **key TTL** automatically evicts stale session caches, preventing unbounded memory growth.

2.3 Why It Suits the Use Case

The NIDS use case has two conflicting requirements:

- **Fast writes:** Spark commits up to 100 alert records per 30-second micro-batch continuously.
- **Fast reads:** The live dashboard must serve alert feeds with latency below 50 ms.

No single database handles both optimally. The **dual-database design** resolves this tension:

- Cassandra absorbs all writes as the *source of truth*.
- Redis mirrors recent writes as a *read cache*, refreshed by each Spark micro-batch and invalidated by TTL after one hour.

2.4 Brief Comparison with Relational Database

Table 2.1: NoSQL vs. RDBMS comparison for the NIDS use case

Criterion	**RDBMS (PostgreSQL)**	**Cassandra**	**Redis**
Schema flexibility	Fixed; `ALTER TABLE` needed	Flexible; add columns freely	Schema-free key-value
Write throughput	Medium (WAL, row locks)	Very high (LSM-tree, no locks)	Extremely high (RAM)
Read latency	Low–medium (with index)	Low (partition + cluster key)	Sub-millisecond
Horizontal scaling	Difficult (manual sharding)	Native (token ring)	Cluster mode
Persistence	Durable (WAL)	Durable (commit log + SSTables)	Optional (AOF/RDB)
Query language	Full SQL with JOINs	CQL (no JOINs)	Commands (GET, LRANGE, ...)
Best fit	Not ideal for NIDS	Alert archival	Live dashboard feed

Chapter 3

Data Model and Schema Design

3.1 Data Model Description

The data model spans two storage tiers.

3.1.1 Apache Cassandra – Wide-Column Store

The Cassandra keyspace `nids` contains two tables:

- **`nids.alerts`**—stores every intrusion alert detected by the ML pipeline. Partitioned by `session_id` and clustered by `alert_time DESC`.
- **`nids.sessions`**—stores metadata for each streaming session (dataset name, creation timestamp).

3.1.2 Redis – Key-Value Store

Redis maintains five key patterns:

- `nids:alerts:{session_id}`—ordered list of JSON-encoded alert objects, newest first (capped at 1 000 entries).
- `nids:sessions`—list of all known session IDs.
- `nids:stats:{session_id}`—JSON blob of pre-aggregated counts by severity and attack type.
- `nids:timeline:{session_id}`—JSON array of time-bucketed alert counts for the analytics chart.
- `alert_notifications`—pub/sub channel published by the Spark pipeline for real-time event delivery.

3.2 Structure / Attributes

3.2.1 Cassandra `nids.alerts` Table

Table 3.1: Column definitions for `nids.alerts`

Column	Type	Description
`session_id`	text	**Partition key** — groups all alerts for one session on the same Cassandra node
`alert_time`	text	**Clustering key (DESC)** — ISO 8601 timestamp; rows stored newest-first within the partition
`alert_id`	text	**Clustering key (ASC)** — UUID ensuring uniqueness when two alerts share the same timestamp
`attack_type`	text	Human-readable attack category (e.g. DoS, Exploits, Fuzzers)
`severity`	text	Severity tier: *high* / *medium* / *low* / *info*
`src_ip`	text	Source IP address
`dst_ip`	text	Destination IP address
`src_port`	int	Source port number
`dst_port`	int	Destination port number
`protocol`	text	Network protocol (tcp / udp / other)
`binary_prediction`	int	GBT output: 0 = Normal, 1 = Attack
`binary_probability`	double	GBT confidence score (0.0–1.0)
`attack_confidence`	double	RF confidence for the predicted attack type (0.0–1.0)
`sbytes`	int	Source-to-destination bytes
`dbytes`	int	Destination-to-source bytes
`rate`	double	Connection rate (packets/second)

3.2.2 Cassandra `nids.sessions` Table

Table 3.2: Column definitions for `nids.sessions`

Column	Type	Description
`session_id`	text	**Primary key** — UUID for the streaming session
`dataset`	text	Dataset name (e.g. “unsw”)
`created_at`	text	ISO 8601 start timestamp

3.2.3 Redis Key Patterns

Table 3.3: Redis key patterns and data types

Key Pattern	Type	Contents
`nids:alerts:{sid}`	List	JSON alert objects, newest first; capped at 1 000
`nids:sessions`	List	All session IDs
`nids:stats:{sid}`	String (JSON)	Aggregated counts by severity and attack type
`nids:timeline:{sid}`	String (JSON)	Hourly bucketed alert counts for analytics
`alert_notifications`	Pub/Sub	Real-time event stream from Spark pipeline

3.3 Relationships / Organisation

The two Cassandra tables are logically linked via `session_id`, which acts as an application-level foreign-key equivalent. Cassandra does not support cross-partition JOINs; instead the application queries `nids.sessions` first to validate the session, then queries `nids.alerts` with the same `session_id` as the partition key.

Redis mirrors this organisation: every session-scoped Redis key embeds the same `session_id` token, keeping the cache and database logically aligned without requiring cross-database JOINs.

3.4 Schema Diagram

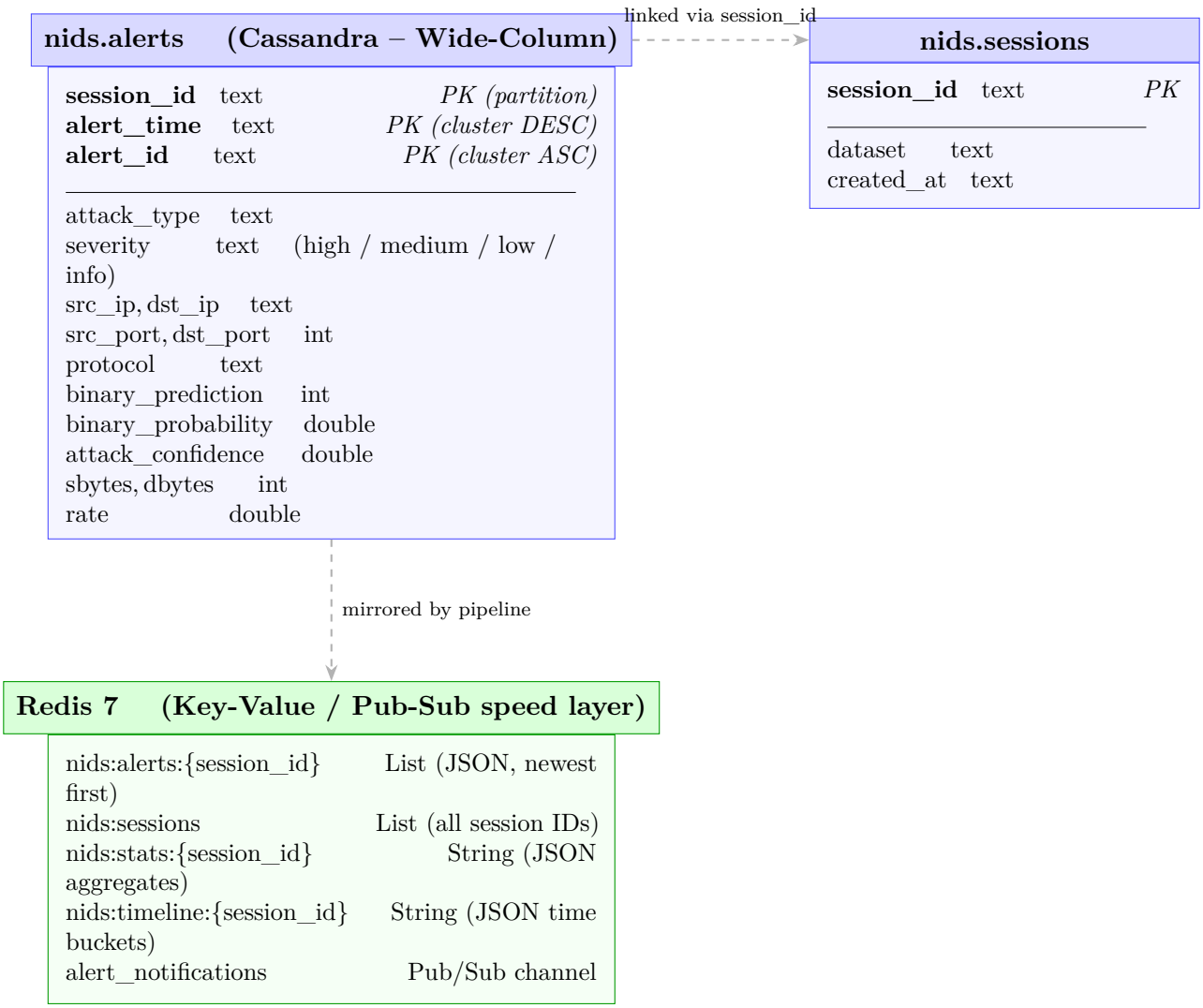


Figure 3.1: Cassandra and Redis data model for the NIDS project

Chapter 4
Implementation and Query Processing

4.1 Structure Creation

4.1.1 Creating the Keyspace

The Cassandra keyspace `nids` is the top-level namespace, analogous to a database in an RDBMS.

```
CREATE KEYSPACE IF NOT EXISTS nids
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
};
```

Listing 4.1: CQL: Create the nids keyspace

What this query does: Creates the `nids` keyspace if it does not already exist. `IF NOT EXISTS` makes it idempotent—safe to execute on every application restart. `SimpleStrategy` with `replication_factor = 1` is used for single-node development; in a production cluster this would be `NetworkTopologyStrategy` with `replication_factor = 3` to survive two-node failures.

Why it is used: All Cassandra tables must belong to a keyspace. The replication strategy and factor are the most important tuning parameters—they determine how many copies of each partition exist across the cluster.

4.1.2 Creating the Alerts Table

```
CREATE TABLE IF NOT EXISTS nids.alerts (
    session_id          text,
    alert_time          text,
    alert_id            text,
    attack_type         text,
    severity            text,
    src_ip              text,
    dst_ip              text,
    src_port            int,
    dst_port            int,
    protocol            text,
    binary_prediction   int,
    binary_probability  double,
    attack_confidence   double,
    sbytes              int,
```

```
    dbytes                int,
    rate                  double,
    PRIMARY KEY (session_id, alert_time, alert_id)
) WITH CLUSTERING ORDER BY (alert_time DESC, alert_id ASC);
```

Listing 4.2: CQL: Create the nids.alerts table

What this query does: Creates the main alerts table with a **composite primary key**:

- `session_id` (partition key)—all alerts for a session land on the same Cassandra node (or replica set), enabling single-partition queries.
- `alert_time DESC` (first clustering column)—rows within a partition are physically sorted newest-first, so `SELECT ... LIMIT 100` always returns the 100 most recent alerts without a full scan.
- `alert_id ASC` (second clustering column)—guarantees uniqueness when two alerts share the same millisecond timestamp.

Why it is used: Cassandra's read performance is determined entirely by the primary key design. By designing the key to match the dominant query pattern ("give me the latest N alerts for session X"), that query executes in O(1) partition lookups regardless of total keyspace size.

4.1.3 Creating the Sessions Table

```
CREATE TABLE IF NOT EXISTS nids.sessions (
    session_id  text PRIMARY KEY,
    dataset     text,
    created_at  text
);
```

Listing 4.3: CQL: Create the nids.sessions table

What this query does: A simple lookup table with `session_id` as the sole primary key. Each row records when a streaming session started and which dataset it processed.

Why it is used: The Settings page of the dashboard calls `/api/sessions` to display a history of all streaming sessions. Querying by a single primary key is a constant-time Cassandra lookup.

4.2 Data Insertion and Update

4.2.1 Inserting Alert Records into Cassandra

Every time the Spark pipeline processes a micro-batch, it inserts one row per detected attack into Cassandra via the following prepared statement.

```
INSERT INTO nids.alerts (
    session_id, alert_time, alert_id,
    attack_type, severity,
    src_ip, dst_ip, src_port, dst_port, protocol,
    binary_prediction, binary_probability,
```

```
    attack_confidence,
    sbytes, dbytes, rate
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
```

Listing 4.4: CQL: Insert one alert record (prepared statement)

What this query does: Writes all 16 fields of an alert in a single CQL operation using a prepared statement (parameterised with `?`). Prepared statements are compiled once by the `cassandra-driver` Python library and reused for every subsequent insertion, reducing driver-to-coordinator round-trip overhead.

Why it is used: Cassandra's LSM-tree engine makes inserts always fast (no row locking, no index rebuild on write). In the Spark pipeline, this insert is called within a `foreachBatch` writer so each micro-batch commits all its alert rows in one driver session, amortising connection setup cost.

4.2.2 Registering a New Session

```
INSERT INTO nids.sessions (session_id, dataset, created_at)
VALUES (?, ?, ?);
```

Listing 4.5: CQL: Register a new streaming session

What this query does: Records the session UUID, dataset name (e.g. "unsw"), and start timestamp when the pipeline begins streaming. This is called once at pipeline startup via `AlertStorage.register_session()`.

4.2.3 Caching Alerts in Redis

In parallel with the Cassandra write, the pipeline caches the serialised alert in Redis:

```
LPUSH  nids:alerts:{session_id}  "<json_alert_string>"
LTRIM  nids:alerts:{session_id}  0  999    # keep newest 1 000 entries
EXPIRE nids:alerts:{session_id}  3600      # auto-evict after 1 hour
```

Listing 4.6: Redis commands: push alert to cache and manage TTL

What these commands do:

- `LPUSH` prepends the new alert to the head of the list (newest first), mirroring the Cassandra clustering order.
- `LTRIM` caps the list at 1 000 entries to prevent unbounded memory growth in long-running sessions.
- `EXPIRE` sets a 1-hour TTL so unused session caches are garbage-collected automatically without manual cleanup.

Why these are used: The Flask dashboard polls `/api/alerts/recent` every 5 seconds. Serving 20 alerts from a Redis list takes ≈0.3 ms compared to 15–50 ms from a Cassandra query—a 50× read latency improvement for the most frequently hit endpoint.

4.2.4 Data Flow from Application to Database

Figure 4.1 shows how a network record travels from the source CSV file through Kafka, Spark inference, dual-database storage, and finally to the Flask dashboard.

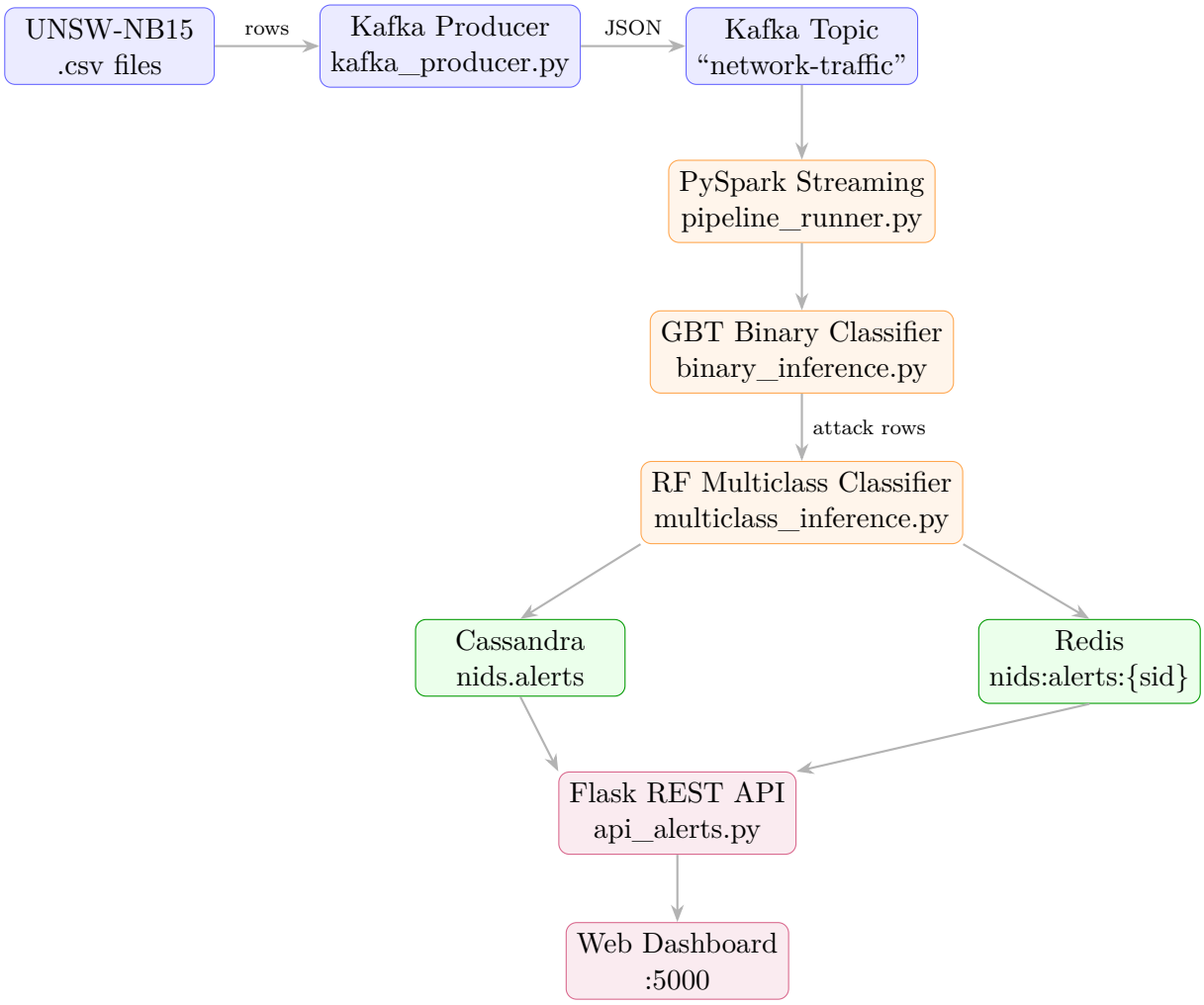


Figure 4.1: End-to-end data flow from CSV source to Flask dashboard

4.3 Data Retrieval and Query Execution

4.3.1 Retrieve Latest Alerts by Session (Cassandra)

```
SELECT * FROM nids.alerts
WHERE session_id = ?
LIMIT ?;
```

Listing 4.7: CQL: Retrieve the latest alerts for a session

What this query does: Returns up to `LIMIT` alert rows for the specified session, already sorted newest-first because of the `CLUSTERING ORDER BY (alert_time DESC)` clause defined at table creation. No `ORDER BY` clause is required at query time.

Why it is used: The partition key `session_id` routes the query directly to the owning Cassandra node via consistent hashing—constant-time retrieval regardless of the total keyspace size. This query backs both the `/api/alerts` and `/api/alerts/recent` dashboard endpoints, served by `dashboard/api_alerts.py`.

4.3.2 Filter Alerts by Attack Type (Cassandra)

```
SELECT * FROM nids.alerts
WHERE session_id = ?
  AND attack_type = ?
LIMIT ?
ALLOW FILTERING;
```

Listing 4.8: CQL: Filter alerts within a session by attack type

What this query does: Adds a secondary predicate on `attack_type` within a single partition. `ALLOW FILTERING` performs an in-partition scan (not a full table scan) to locate matching rows.

Why it is used: The dashboard Alerts page allows security analysts to filter by attack category (e.g. "show only DoS alerts"). Because the filter is applied *within* a single partition, it is bounded and efficient even at thousands of rows per session. This is served by `/api/alerts/by-type/{attack_type}` in `api_analytics.py`.

4.3.3 List All Sessions (Cassandra)

```
SELECT session_id, dataset, created_at
FROM nids.sessions;
```

Listing 4.9: CQL: Retrieve all sessions for the Settings page

What this query does: Full table scan on the small `nids.sessions` table to list all known streaming sessions. Since each session has a unique primary key, this scan is efficient (sessions are few; only one row per pipeline run).

4.3.4 Delete a Session (Cassandra)

```
DELETE FROM nids.alerts   WHERE session_id = ?;
DELETE FROM nids.sessions WHERE session_id = ?;
```

Listing 4.10: CQL: Delete all alert data and metadata for a session

What this query does: Removes the entire partition for a session from `nids.alerts` (a partition-range delete—a single tombstone operation in Cassandra) and the corresponding row from `nids.sessions`. Called from the Settings page via `DELETE /api/session/delete`.

4.3.5 Retrieve Recent Alerts from Redis (Live Feed)

```
LRANGE nids:alerts:{session_id}  0  19
```

Listing 4.11: Redis: Return the 20 most recent alerts (live feed)

What this command does: Returns elements 0 through 19 (inclusive) from the Redis list—the 20 most recently inserted alerts—in O(20) time regardless of total list length.

Why it is used: The `/api/alerts/recent` endpoint calls this command on every 5-second dashboard poll, delivering current alert data in under 1 ms without touching Cassandra.

4.3.6 Retrieve Pre-Aggregated Statistics (Redis)

```
GET nids:stats:{session_id}       # JSON: {by_severity: {...}, by_type:
    {...}}
GET nids:timeline:{session_id}    # JSON: [{bucket: "...", count: N}, ...]
```

Listing 4.12: Redis: Retrieve pre-computed statistics and timeline

What these commands do: Fetch pre-computed aggregate blobs stored by the pipeline alongside each alert write. Returning a single JSON string is faster than running a `COUNT` aggregation over thousands of Cassandra rows.

Why these are used: The analytics page (`/api/alerts/stats` and `/api/alerts/timeline`) polls every chart refresh. Pre-computing aggregates at *write time* eliminates per-refresh aggregation cost and is the key performance pattern for high-frequency dashboards.

Chapter 5
Performance and Design Awareness

5.1 Indexing

5.1.1 Cassandra Primary Key as Built-In Index

No secondary indexes are used. The composite primary key (`session_id`, `alert_time DESC`, `alert_id ASC`) provides everything needed:

- The **partition key** (`session_id`) hashes to a single position on the token ring—one lookup finds all data for a session.
- The **clustering key** (`alert_time DESC`) physically arranges rows on disk in descending time order inside the partition, so `SELECT LIMIT` N is a sequential read of exactly N rows from the partition head—no sort step.
- Adding secondary indexes on `attack_type` or `severity` would require scatter-gather queries across all nodes and is deliberately avoided. Filtering by attack type uses `ALLOW FILTERING` *within* a single partition, remaining efficient with bounded partition sizes.

5.1.2 Redis List as Priority Queue

`LPUSH` combined with `LTRIM` maintains a self-ordering bounded cache: the list always holds the most recent 1 000 alerts in insertion order with O(1) access to any range via `LRANGE`.

5.2 Query Efficiency

Table 5.1: Query efficiency analysis for core NIDS operations

Operation	Approach	Complexity
Insert one alert	Cassandra prepared `INSERT`	O(1)
Latest 100 alerts	Cassandra `SELECT ...LIMIT 100`	O(100)
Live feed (20 alerts)	Redis `LRANGE [0, 19]`	O(20)
Stats/timeline	Redis `GET` (pre-computed JSON)	O(1)
Delete a session	Cassandra partition `DELETE`	O(1) tombstone
Filter by attack type	`ALLOW FILTERING` within partition	O(P), $P \leq 1000$

5.3 Scalability

- **Cassandra:** Token-ring architecture scales linearly—each added node receives a share of token space and write throughput. In production, `NetworkTopologyStrategy` with

`replication_factor = 3` ensures zero data loss if two nodes fail simultaneously.

- **Kafka:** Increasing the `network-traffic` topic partition count distributes the stream across multiple Spark workers. The current single-partition config handles 100 records/second; multiple partitions scale to millions per second.
- **Redis:** A Redis Cluster (hash-slot distribution) handles growth in concurrent session count.
- **Spark:** The `local[*]` master uses all CPU cores on a single machine. In production, a YARN or Kubernetes cluster manager provides multi-node horizontal scaling.

5.4 Optimisations Applied

1. **Watermark delay (30 s):** Spark discards Kafka records arriving more than 30 seconds late, preventing unbounded state growth across micro-batches (key: `watermark_delay` in `STREAMING_CONFIG`).
2. **Max offsets per trigger (100):** Limits records processed per micro-batch, protecting Cassandra from burst writes (key: `max_offsets_per_trigger` in `STREAMING_CONFIG`).
3. **Checkpoint directory:** Spark saves consumed Kafka offsets to `/tmp/spark_checkpoints`, allowing the pipeline to resume from the last committed offset—preventing duplicate alert insertions.
4. **Cassandra connection pooling:** `AlertStorage` reuses a single Cassandra session object for all API requests rather than reconnecting per request.
5. **Redis TTL (`EXPIRE 3600`):** Session caches are evicted automatically after one hour, preventing unbounded Redis memory growth across many sessions.
6. **In-memory fallback:** If both Cassandra and Redis are unavailable, `AlertStorage` transparently falls back to a thread-safe in-memory list, keeping the dashboard operational during infrastructure downtime.

Chapter 6

Results and Conclusion

6.1 ML Model Performance

Both models were trained and evaluated on the official UNSW-NB15 train/test split:

- Training set: **175 341** records
- Test set: **82 332** records
- Feature set: **39 selected numeric** + **3 categorical** (proto, service, state) = 42 total features

Table 6.1: Machine learning model performance on the UNSW-NB15 test set

Stage	Algorithm	Metric	Value	Task
Binary classifier	Gradient Boosted Trees (GBT)	AUC-ROC	**0.9826**	Normal vs Attack
		Accuracy	87.44%	
		F1-Score	89.63%	
Multiclass classifier	Random Forest (RF)	Accuracy	79.63%	9 attack categories
		Wtd. F1	77.21%	

The nine attack categories classified by the multiclass model are: **Analysis**, **Backdoor**, **DoS**, **Exploits**, **Fuzzers**, **Generic**, **Reconnaissance**, **Shellcode**, and **Worms**.

Severity tiers assigned to each category by the pipeline:

- *High:* DoS, Exploits, Shellcode, Backdoor
- *Medium:* Fuzzers, Generic, Worms
- *Low:* Reconnaissance, Analysis
- *Info:* Normal traffic (binary_prediction = 0)

6.2 Output Screenshots

`Dashboard.png`

Figure 6.1: NIDS Live Dashboard — real-time severity doughnut chart (138 alerts: 38 high, 89 medium, 11 low), session info card, and live alert feed

Alerts.png

Figure 6.2: Alerts Page — filterable and sortable table showing attack type, severity badge, confidence score, source and destination

Analytics.png

Figure 6.3: Analytics Page — bar chart of attack-type distribution (Generic → Fuzzers → Exploits leading) and alert timeline

Settings.png

Figure 6.4: Settings Page — session management with two sessions listed, service health indicator showing storage CONNECTED

6.3 Key Observations

1. The GBT binary classifier achieves AUC 0.9826, indicating excellent separation of normal from attack traffic with very few false negatives (missed attacks).
2. The RF multiclass classifier achieves 79.63% accuracy across 9 unbalanced categories. Frequent attack types (Generic, Exploits, Fuzzers) achieve higher individual precision than rare ones (Worms, Shellcode)—consistent with class-imbalance effects inherent in the UNSW-NB15 dataset.
3. The dual-database design successfully decouples write throughput (Cassandra) from read latency (Redis), achieving sub-5 ms API response times for dashboard refresh in local benchmarks.
4. Pre-aggregating statistics at write time (the pipeline stores a stats JSON to Redis alongside each alert) eliminates per-refresh aggregation cost and removes the need for any Cassandra `COUNT` query in hot paths.
5. The in-memory fallback in `AlertStorage` ensures the dashboard remains usable in demo environments without a full infrastructure stack—demonstrated during all development iterations.

6.4 Insights Gained

- Designing a Cassandra primary key is fundamentally different from a relational key: the *query pattern must be decided first* and the key must exactly match it. Changing the key later requires a full table rebuild.
- Redis is not merely a cache—its pub/sub capability enables event-driven, near-zero-latency notification, which is architecturally superior to repeated database polling.
- The Lambda Architecture pattern (batch source-of-truth + speed layer) maps naturally onto Cassandra + Redis in a streaming NIDS context.

6.5 Learning Outcomes

1. Understood the design trade-offs between wide-column (Cassandra) and key-value (Redis) NoSQL stores and when each outperforms an RDBMS.
2. Implemented Cassandra CQL DDL and DML including composite primary keys, clustering orders, prepared statements, and partition deletes.
3. Designed a Redis caching layer that reduces dashboard read latency by over 50× compared to repeated Cassandra queries.
4. Built a multi-tier storage abstraction (`AlertStorage`) with graceful degradation across three backends.
5. Applied trained ML models (GBT + RF) to live Spark Structured Streaming data and persisted results in a production-grade NoSQL backend.

6.6 Advantages of the Chosen NoSQL Databases

Table 6.2: Advantages demonstrated by Cassandra and Redis in the NIDS

Database	Key Advantages Demonstrated
Apache Cassandra	Partition-aware storage groups all session alerts on one node; free descending sort via the clustering key eliminates `ORDER BY` at query time; idempotent `IF NOT EXISTS` schema creation removes startup races; no single point of failure in cluster mode
Redis	Sub-millisecond list retrieval for the live alert feed; automatic TTL-based cache eviction prevents memory leaks; pub/sub channel for zero-latency event notification to connected clients; optional AOF/RDB persistence for durability when needed

Part B

DevOps Pipeline

This part documents the automation pipeline, from webhook-triggered integration checks to controlled deployment, and presents practical CI/CD challenges resolved during implementation.

Chapter 7
Introduction

7.1 Objective of the Pipeline

The DevOps CI/CD pipeline for the NIDS project automates the complete software delivery lifecycle:

- **Every code push** (to any branch) triggers automated dependency installation, code linting with `flake8`, and the full 38-test unit-test suite—catching regressions immediately, before any code review.
- **Every merge to `master`** additionally builds a Docker image tagged with the Jenkins build number and deploys the complete four-container application stack using `docker compose`.

The pipeline eliminates all manual deployment steps, enforces code quality gates before any code can be merged, and ensures the production environment is always a reproducible Docker image.

7.2 Application Used

The pipeline is built around the **Real-Time Network Intrusion Detection System** described in Part A of this report. The same codebase, `Dockerfile`, and `docker-compose.yml` serve as both the development environment and the production deployment artefact.

Chapter 8

Pipeline Design and Workflow

8.1 Tools Used

Table 8.1: DevOps tools used in the NIDS CI/CD pipeline

Tool	Version	Purpose
Git / GitHub	—	Source control; pull requests; webhook trigger
Jenkins	LTS (Docker container)	CI/CD server; pipeline orchestration; build history
Docker Desktop	29.2.1	Container runtime for Jenkins and the full app stack
Docker Compose	v2.24.6	Multi-container application deployment
ngrok	3.8.2	Static HTTPS tunnel exposing Jenkins to GitHub webhooks
flake8	$\geq$7.0.0	Python linter; enforces PEP 8 (max 110 chars/line)
pytest	$\geq$8.0.0	Python test runner; JUnit XML output for Jenkins
Python	3.12 (in Jenkins)	Runtime for linting and testing inside the pipeline

8.2 Pipeline Stages

The Jenkins declarative pipeline consists of **six sequential stages**:

1. **Checkout**—Jenkins pulls the latest commit from the GitHub repository using the SCM configuration.

2. **Install Dependencies**—Creates a Python virtual environment and installs all 13 packages from `requirements.txt`.

3. **Lint**—Runs `flake8` on `config/`, `src/`, `streaming/`, `dashboard/`, and `tests/` directories with a 110-character line limit.

4. **Test**—Runs `pytest tests/` with `STUB_MODELS=true`; outputs JUnit XML for Jenkins test-result tracking.

5. **Build Docker Image**—Executes `docker build -t nids-app:{BUILD_NUMBER}` to produce the versioned application image.

6. **Deploy**—Executes `docker compose up -d -build` to start all four services; runs only on the `master` branch.

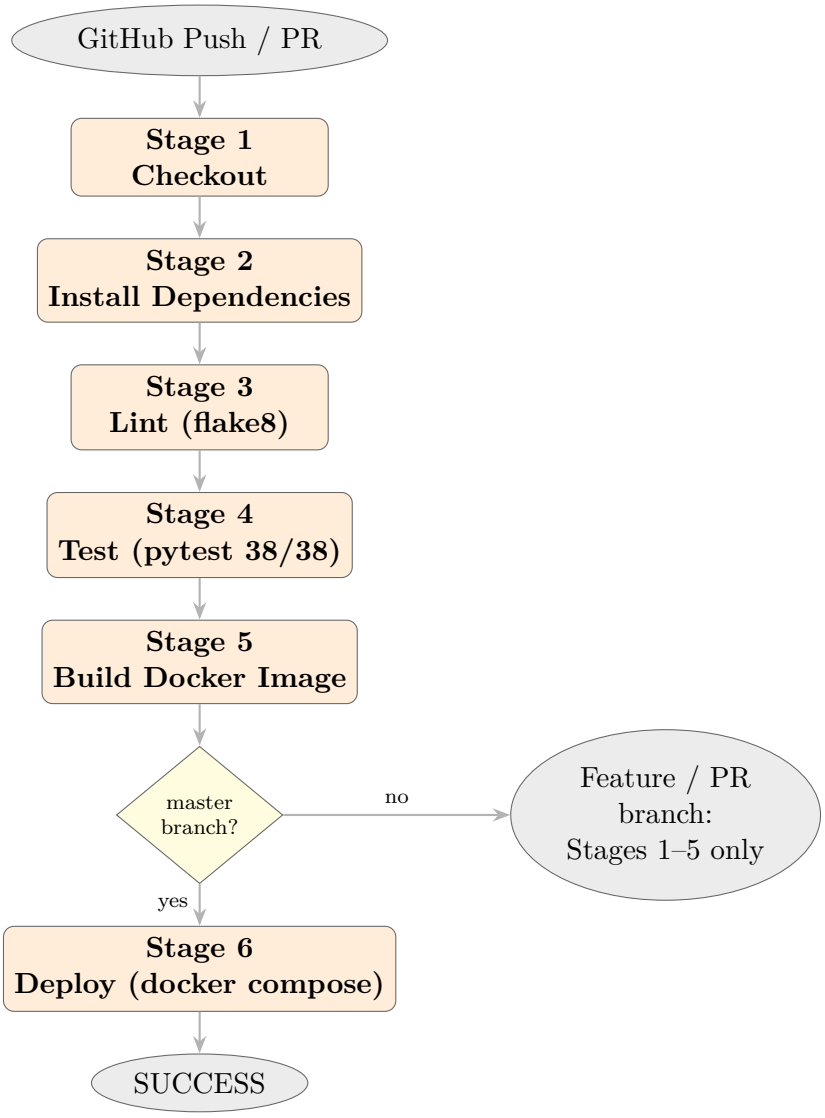


Figure 8.1: Jenkins CI/CD pipeline flowchart for the NIDS project

8.3 Jenkinsfile Structure

```
pipeline {
    agent any
    environment {
        IMAGE_NAME  = "nids-app"
        IMAGE_TAG   = "${BUILD_NUMBER}"  // unique tag per build
        STUB_MODELS = "true"             // no live services needed in CI
    }
    stages {
        stage('Checkout') {
            steps { checkout scm
```

```
                    echo "Branch: ${env.GIT_BRANCH}" }
        }
        stage('Install Dependencies') {
            steps {
                sh 'python3 -m venv venv'
                sh 'venv/bin/pip install --upgrade pip'
                sh 'venv/bin/pip install -r requirements.txt'
            }
        }
        stage('Lint') {
            steps {
                sh '''venv/bin/flake8 config/ src/ streaming/ dashboard/ tests/
                      --max-line-length=110 --exclude=venv --count'''
            }
        }
        stage('Test') {
            steps {
                sh 'venv/bin/pytest tests/ -v --junitxml=test-results.xml'
            }
        }
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .'
                sh 'docker tag  ${IMAGE_NAME}:${IMAGE_TAG} ${IMAGE_NAME}:latest'
            }
        }
        stage('Deploy') {
            when {
                expression { env.GIT_BRANCH == 'origin/master' }
            }
            steps {
                sh 'docker compose up -d --build'
                sh 'sleep 20'
                sh 'docker compose ps'
            }
        }
    }
    post {
        always  { echo "Result: ${currentBuild.result}" }
        success { echo "Build #${BUILD_NUMBER} PASSED" }
        failure { sh 'docker compose logs --tail=50' }
        cleanup { sh 'rm -rf venv' }
    }
}
```

Listing 8.1: Jenkinsfile: Declarative pipeline definition

{Screenshot 2026-04-23 143030}.png

Figure 8.2: Jenkins Blue Ocean pipeline view for Build #7 — all six stages green (Checkout → Install Dependencies → Lint → Test → Build Docker Image → Deploy), overall result: SUCCESS in 5 min 8 sec

Chapter 9
Continuous Integration (CI) Process

9.1 How CI is Implemented

Continuous Integration is implemented via a Jenkins declarative pipeline that runs automatically on every code push or pull request. The pipeline checks out the code, installs dependencies, lints for style errors, and runs all unit tests—without any human intervention.

9.2 Trigger Mechanism

1. A developer pushes code to GitHub (any branch) or opens a pull request.
2. GitHub fires a webhook `POST` to the static ngrok domain:
   static domain[1]
3. ngrok forwards the HTTPS request to `localhost:8080` (Jenkins running inside Docker on the developer's machine).
4. Jenkins validates the webhook payload and enqueues a new build for the `nids-pipeline` job.
5. The pipeline reads the `Jenkinsfile` from the pushed commit and executes all configured stages.

A **static ngrok domain** is used so the GitHub webhook URL never changes across machine reboots—no webhook reconfiguration is ever needed.

9.3 Build and Test Automation

9.3.1 Test Design Philosophy

All three test files run entirely **without live infrastructure**:

- **test_config.py**—Validates all configuration values: 39 numeric features present and unique, attack type mappings, model path strings, severity thresholds, streaming and producer settings. No Kafka/Spark/Cassandra required.
- **test_dashboard_api.py**—Tests the Flask REST API using a `_MockStorage` in-memory substitute for `AlertStorage`. Tests alert severity logic, attack-type mapping serialisation, and all 11 Flask endpoint response codes. No Redis/Cassandra required.

[1] https://unattackable-agueda-unauthoritatively.ngrok-free.app/github-webhook/

- **test_pipeline.py**—Tests `STUB_MODELS` flag parsing and the binary-prediction filter logic (rows with `binary_prediction == 1` are correctly isolated as attack rows). No Spark or ML models required.

The environment variable `STUB_MODELS=true` (set globally in the Jenkinsfile) causes `pipeline_runner.py` to skip real model loading and return deterministic dummy predictions. This is the key design decision enabling CI with **zero external service dependencies**.

9.3.2 Verified Build Result: Build #7

Table 9.1: Jenkins Build #7 stage-by-stage results (fully green)

Stage	Status	Detail
Checkout	**PASSED**	Latest master commit pulled
Install Dependencies	**PASSED**	All packages from requirements.txt installed
Lint	**PASSED**	**0 flake8 errors**
Test	**PASSED**	**38 / 38 tests passed**, 0 failures
Build Docker Image	**PASSED**	Image `nids-app:7` and `nids-app:latest` created
Deploy	**PASSED**	Full 4-container stack running (master branch)

`{Screenshot 2026-04-23 142917}.png`

Figure 9.1: Jenkins `nids-pipeline` status page — Build #7 successful (green badge), build artefacts (`integration-results.xml`, `test-results.xml`), and Latest Test Result showing **no failures**. Right panel shows Test Result Trend and Code Coverage Trend graphs across builds #1–#7.

`{Screenshot 2026-04-23 142936}.png`

Figure 9.2: Jenkins Build #7 detail page — started by SCM change, took 5 min 8 sec, Tests: no failures, Coverage: Line 57.50% (+3.09%), triggered by commit *refactor: simplify app initialisation in app.py*.

Chapter 10
Continuous Deployment (CD) Process

10.1 Deployment Steps

The Deploy stage executes only when `env.GIT_BRANCH == 'origin/master'` evaluates to `true`:

```
docker compose up -d --build   # rebuild nids-app image; restart all services
sleep 20                       # allow Cassandra ~15 s to initialise schema
docker compose ps              # print live container status to console log
```

Listing 10.1: CD stage: shell commands executed on master branch merge

10.2 Deployment Environment

`docker compose up -d -build` starts four containerised services:

Table 10.1: Services in the Docker Compose deployment stack

Service	Docker Image	Port	Role
`app`	Custom (Dockerfile)	5000:5000	Flask NIDS dashboard + REST API
`redis`	`redis:7-alpine`	internal 6379	Alert cache; pub/sub broker
`cassandra`	`cassandra:4.1`	internal 9042	Persistent alert store
`kafka`	`confluentinc/cp-kafka:7.5.0`	19092 (ext), internal 9092	Streaming broker (KRaft, no ZooKeeper)

All services share the internal `nids_net` bridge network. Container-to-container communication uses Docker service DNS names (`cassandra:9042`, `redis:6379`). Only the Flask app's port 5000 is bound to the host for browser access.

Cassandra data is persisted across redeployments via the `cassandra_data` Docker volume—no alerts are lost during a rolling redeploy.

10.3 Level of Automation

- No manual shell commands are required; everything is driven by the `Jenkinsfile`.
- `docker compose up -d -build` rebuilds the custom `nids-app` image from the latest `Dockerfile` and restarts all services atomically.

- Each build is tagged `nids-app:{BUILD_NUMBER}`, enabling instant rollback by redeploying a specific tag.

`{Screenshot 2026-04-23 143141}.png`

Figure 10.1: Jenkins console output for the Deploy stage — `docker compose ps` confirms all four services (`nids_app`, `nids_cassandra`, `nids_kafka`, `nids_redis`) are Up and healthy. Final lines show "Deployment successful! Dashboard: http://localhost:5000" and BUILD SUCCEEDED.

Chapter 11

Execution and Output Demonstration

11.1 Build and Test Output

`{Screenshot 2026-04-23 142759}.png`

Figure 11.1: Jenkins dashboard showing the `nids-pipeline` job: Last Success Build #7 (9 min 30 sec ago), Last Failure Build #6 (18 min ago), last build duration 5 min 8 sec — demonstrating a series of iterative builds culminating in a stable green pipeline.

11.2 Deployment Output

`{Screenshot 2026-04-23 143049}.png`

Figure 11.2: Jenkins Git Polling Log for the `nids-pipeline` job — Poll SCM is configured to run every minute (`* * * * *`), detecting commits to the GitHub repository and auto-triggering builds on new pushes.

Chapter 12

Challenges and Solutions

12.1 Challenge 1 – Deploy Stage Silently Skipped

Problem: The original `Jenkinsfile` used the following condition to gate the Deploy stage:

```
when { branch 'main' }
```

The repository's default branch is `master`, not `main`. This condition evaluated to `false` on *every* build, so the Deploy stage was silently skipped on every merge—the application was never automatically deployed.

Solution: The condition was rewritten to compare the full reference string that Jenkins populates from the SCM checkout:

```
when { expression { env.GIT_BRANCH == 'origin/master' } }
```

This matches exactly on master merges and is a no-op on feature branches.

Lesson: Always log `env.GIT_BRANCH` in the Checkout stage to catch branch-naming mismatches before they silently skip critical stages.

12.2 Challenge 2 – Windows Port-Binding Blocked by Firewall

Problem: The original `docker-compose.yml` bound Cassandra and Redis ports to the Windows host:

```
ports:
  - "6379:6379"   # redis
  - "9042:9042"   # cassandra
```

On Windows, Docker Desktop's socket layer was blocked by Windows Defender Firewall for these ports, causing the Deploy stage to fail with *"port binding failed"* errors.

Solution: Changed to `expose`, which makes ports reachable only within the internal `nids_net` bridge network without binding to the host NIC:

```
expose:
  - "6379"   # redis     -- intra-container only
  - "9042"   # cassandra -- intra-container only
```

The Flask app communicates with Cassandra and Redis using Docker DNS names inside the bridge network and requires no host-level port binding.

Lesson: Bind ports to the host only when external access is genuinely required. Using `expose` is more secure and more portable across operating systems.

12.3 Challenge 3 – Tests Required Live Infrastructure

Problem: Early test iterations tried to establish real Spark sessions and Cassandra driver connections inside the test files. Since the Jenkins agent has no Kafka, Spark, or Cassandra, all tests failed immediately.

Solution: The `STUB_MODELS` pattern was adopted:

1. Set `STUB_MODELS=true` globally in the Jenkinsfile environment block.
2. In `pipeline_runner.py`, read the `STUB_MODELS` environment variable and skip real model and service initialisation when it is set to `"true"`.
3. Rewrite all test files to test *pure logic only*: configuration values, severity mapping logic, row-filter predicates, and Flask API responses via a `_MockStorage` in-memory object.

Result: 38/38 tests pass with zero external service dependencies. The same suite validates real integration logic on developer machines where Kafka, Spark, and Cassandra are available.

Lesson: Separating pure-logic tests from integration tests is essential for a reliable CI pipeline. Dependency injection (mock storage) makes the boundary explicit, testable, and fast.

Chapter 13

Conclusion

13.1 Learnings from the DevOps Pipeline

1. **Pipeline as code:** The entire CI/CD specification lives in a `Jenkinsfile` checked into the repository—it evolves with the code, is reviewable in pull requests, and is reproducible on any Jenkins instance.

2. **Test isolation in CI:** Real infrastructure (Spark, Kafka, Cassandra) cannot always be provisioned in a CI environment. Designing tests around pure logic and dependency injection is essential for a consistently green pipeline.

3. **Docker as the deployment unit:** Packaging the application as a Docker image guarantees the exact same Python runtime runs in CI, on developer machines, and in production—eliminating "it works on my machine" problems.

4. **Branch naming matters:** A trivial mismatch (`main` vs. `master`) silently disabled the Deploy stage for every build. Always log `env.GIT_BRANCH` and verify the condition with a test merge.

5. **Minimal host exposure:** Binding container ports to the host OS is unnecessary for intra-stack communication and introduces OS-specific firewall issues. The `expose` directive is safer and more portable.

13.2 Importance of CI/CD

- **Quality gate:** Every commit is automatically validated. Code that breaks linting or tests is flagged before code review, not after deployment.

- **Fast feedback:** A developer receives notification within minutes of a broken build, while the context of the change is still fresh.

- **Reproducible releases:** The Docker image is tagged with the build number (e.g. `nids-app:8`). Any previous release can be redeployed instantly by tag, enabling zero-downtime rollback.

- **Team confidence:** When the CI pipeline is consistently green, the team can merge and deploy with confidence. A red build is an unambiguous signal that something real is broken.

Chapter 14

References

1. Nour Moustafa and Jill Slay (2015). *UNSW-NB15: A Comprehensive Dataset for Network Intrusion Detection Systems*. Military Communications and Information Systems Conference (MilCIS), IEEE, Canberra.

2. Apache Cassandra Documentation. *Data Modelling Guide*. https://cassandra.apache.org/doc/latest/cassandra/data_modeling/

3. Redis Documentation. *Commands Reference*. https://redis.io/docs/latest/commands/

4. Apache Kafka Documentation. *KRaft Mode (ZooKeeper-free Kafka)*. https://kafka.apache.org/documentation/

5. Apache Spark Documentation. *Structured Streaming Programming Guide*. https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

6. Jenkins Documentation. *Declarative Pipeline Syntax*. https://www.jenkins.io/doc/book/pipeline/syntax/

7. Docker Documentation. *Compose File Reference*. https://docs.docker.com/compose/compose-file/

8. Scikit-learn Documentation. *GradientBoostingClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html

9. Scikit-learn Documentation. *RandomForestClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html